



DOCUMENTACION

SISTEMA DE RESERVAS DE SALAS SAN SEBASTIAN

1. INTRODUCCION:

El desarrollo del **frontend del Sistema de Reservas de Salas San Sebastian**, es una aplicación web administrativa creada para gestionar usuarios, salas y reservas dentro de una empresa o institución.

El sistema fue desarrollado utilizando **React.js, Vite, JavaScript, JSX, Bootstrap y React Router DOM**. Su objetivo principal es entregar una interfaz visual, funcional y responsiva que permita consumir los endpoints del backend desarrollado previamente.

El frontend se conecta con una API RESTful construida con **Node.js, Express, MySQL, Sequelize ORM** y autenticación mediante **JWT**, permitiendo realizar operaciones protegidas como inicio de sesión, gestión de usuarios, gestión de salas y administración de reservas. El backend ya cuenta con endpoints independientes para autenticación, usuarios, salas y reservas.

La aplicación fue organizada mediante páginas y componentes, separando las vistas principales del sistema para facilitar su comprensión, navegación y mantenimiento.

2. OBJETIVO DEL FRONTED

El objetivo del frontend es construir una interfaz web administrativa que permita interactuar con el backend del sistema de reservas de salas, entregando una experiencia clara, ordenada y funcional para el usuario administrador.

Los objetivos principales del frontend son:

- Implementar una pantalla de inicio de sesión funcional.
- Consumir el endpoint de autenticación del backend.
- Guardar el token JWT entregado al iniciar sesión.
- Proteger el acceso a los módulos principales mediante token.
- Gestionar usuarios activos e inactivos.
- Gestionar salas disponibles y no disponibles.
- Crear y administrar reservas de salas.
- Cambiar el estado de las reservas.
- Utilizar una navegación organizada mediante React Router DOM.
- Aplicar diseño responsivo mediante Bootstrap.
- Mantener una estructura de carpetas clara y ordenada.

3. DESCRIPCION GENERAL DEL SISTEMA

El sistema corresponde a un **panel administrativo de reservas de salas**. Su finalidad es permitir que una empresa o institución pueda llevar un control ordenado sobre los espacios físicos disponibles, los usuarios registrados y las reservas realizadas.

El sistema se organiza en tres conceptos principales:

El sistema corresponde a un **panel administrativo de reservas de salas**. Su finalidad es permitir que una empresa o institución pueda llevar un control ordenado sobre los espacios físicos disponibles, los usuarios registrados y las reservas realizadas.

El sistema se organiza en tres conceptos principales:

Usuarios → Personas o funcionarios registrados en el sistema.
Salas → Espacios físicos disponibles para ser reservados.
Reservas → Uso de una sala en una fecha y horario determinado.

El frontend permite que un usuario autorizado ingrese al sistema y administre los distintos módulos desde una interfaz centralizada.

El frontend permite que un usuario autorizado ingrese al sistema y administre los distintos módulos desde una interfaz centralizada.

4. TECNOLOGIAS UTILIZADAS

Tecnología	Uso dentro del proyecto
React.js	Librería principal para construir la interfaz del usuario.
Vite	Herramienta utilizada para crear y levantar el proyecto React.
JavaScript	Lenguaje utilizado para la lógica del frontend.
JSX	Sintaxis utilizada en React para construir las vistas.
Bootstrap	Framework utilizado para diseño responsivo y componentes visuales.
React Router DOM	Librería utilizada para manejar la navegación entre páginas.
Fetch API	Herramienta utilizada para consumir los endpoints del backend.
HTML	Estructura base representada mediante JSX.
CSS	Estilos generales del proyecto.
LocalStorage	Almacenamiento local del token JWT y datos básicos del usuario autenticado.

5. ESTRUCTURA DEL PROYECTO

a. Carpetas principales del frontend

Carpeta	Función dentro del proyecto
reserva_salas2_front/	Carpeta raíz que contiene todo el proyecto frontend desarrollado en React con Vite.
public/	Carpeta generada por Vite para almacenar archivos públicos o estáticos. En este proyecto se mantuvo como parte de la estructura base.
src/	Carpeta principal donde se encuentra el código fuente del frontend.
src/assets/	Carpeta utilizada para guardar recursos visuales del sistema, como el logo institucional, imágenes o íconos que apoyan el diseño de la interfaz.
src/components/	Carpeta destinada a componentes reutilizables del sistema.
src/pages/	Carpeta que contiene las páginas principales del sistema.
src/services/	En esta versión, las peticiones `fetch` se mantuvieron dentro de cada página principal para facilitar la implementación, prueba y comprensión del flujo completo.
node_modules/	Carpeta que contiene las dependencias instaladas mediante npm. No se modifica manualmente.

b. COMPONENTES Y PAGINAS PRINCIPALES

Archivo	Función dentro del proyecto
src/assets/logo.png	Logo institucional utilizado en la pantalla de inicio de sesión para entregar una presentación más corporativa.
src/components/Navbar.jsx	Componente que contiene el menú principal de navegación entre las páginas del sistema.
src/pages/Login.jsx	Página de inicio de sesión. Permite autenticar al usuario y guardar el token JWT.
src/pages/Dashboard.jsx	Panel principal del sistema. Presenta accesos rápidos a los módulos de usuarios, salas y reservas.
src/pages/Usuarios.jsx	Página que permite crear, listar, editar y eliminar lógicamente usuarios activos.
src/pages/UsuariosInactivos.jsx	Página que muestra los usuarios eliminados lógicamente, manteniendo trazabilidad histórica.
src/pages/Salas.jsx	Página que permite crear, listar, editar y eliminar lógicamente salas.
src/pages/Reservas.jsx	Página que permite crear, listar y cambiar el estado de las reservas.

c. ARCHIVOS BASE Y CONFIGURACION

Archivo	Función dentro del proyecto
src/App.jsx	Define las rutas principales del frontend mediante React Router DOM.
src/main.jsx	Punto de entrada de la aplicación. Carga React, App, BrowserRouter y Bootstrap.
src/index.css	Archivo de estilos globales del proyecto.
package.json	Contiene la información del proyecto, scripts y dependencias instaladas.
package-lock.json	Registra las versiones exactas de las dependencias instaladas.
eslint.config.js	Archivo de configuración base de ESLint generado por Vite para revisión de código JavaScript y React.
index.html	Archivo HTML base donde React monta la aplicación mediante el elemento root.
vite.config.js	Archivo de configuración de Vite.
README.md	Archivo de documentación general del frontend, instalación, uso, rutas y estructura del proyecto.
.gitignore	Define qué archivos o carpetas no deben subirse a un repositorio, como node_modules.

6. DESCRIPCION DE CARPETAS PRINCIPALES

- **src/assets:**

En este proyecto se utiliza para guardar el **logo institucional del sistema**, el cual se incorpora en la pantalla de inicio de sesión para entregar una presentación más corporativa y profesional.

- **src/components**

La carpeta src/components contiene componentes reutilizables del sistema. Actualmente contiene el componente:

Navbar.jsx

Este archivo corresponde al menú principal de navegación, permitiendo acceder a las distintas páginas del sistema, como Dashboard, Usuarios, Inactivos, Salas y Reservas

- **src/pages**

La carpeta src/pages contiene las páginas principales del frontend.

Estas páginas son:

Login.jsx
Dashboard.jsx
Usuarios.jsx
UsuariosInactivos.jsx
Salas.jsx
Reservas.jsx

Cada archivo representa una sección funcional del sistema.

- **src/services**

En esta versión, las peticiones fetch se mantuvieron dentro de cada página principal para facilitar la implementación, prueba y comprensión del flujo completo.

- **public**

La carpeta public es generada por Vite y sirve para almacenar archivos públicos o estáticos.

En este proyecto se mantuvo como parte de la estructura base, aunque no fue utilizada directamente para los recursos principales.

7. ARCHIVO eslint.config.js

El archivo eslint.config.js fue generado automáticamente por Vite al crear el proyecto React.

Durante el desarrollo del proyecto, este archivo se mantuvo con la configuración original generada por Vite y no fue necesario modificarlo. El desarrollo del frontend se realizó principalmente en los componentes y páginas ubicados dentro de la carpeta src.

8. RUTAS IMPLEMENTADAS EN EL FRONTEND

El frontend fue organizado mediante rutas utilizando **React Router DOM**, permitiendo la navegación entre las distintas páginas del sistema.

Ruta	Página asociada	Función
/login	Login.jsx	Permite iniciar sesión con correo y contraseña.
/dashboard	Dashboard.jsx	Muestra el panel principal del sistema.
/usuarios	Usuarios.jsx	Permite gestionar usuarios activos.
/usuarios-inactivos	UsuariosInactivos.jsx	Permite visualizar usuarios eliminados lógicamente.
/salas	Salas.jsx	Permite gestionar salas disponibles o no disponibles.
/reservas	Reservas.jsx	Permite crear, listar y cambiar estados de reservas.

9. ENDPOINTS DEL BACKEND CONSUMIDOS POR EL FRONTEND

El frontend consume endpoints independientes del backend mediante fetch, enviando el token JWT en las rutas protegidas. El backend cuenta con módulos de autenticación, usuarios, salas y reservas, utilizando métodos HTTP como GET, POST, PUT, PATCH y DELETE.

Módulo	Método	Endpoint	Uso en el frontend
Auth	POST	http://localhost:3000/api/auth/login	Iniciar sesión y obtener token JWT.
Usuarios	GET	http://localhost:3000/api/usuarios	Listar usuarios.
Usuarios	POST	http://localhost:3000/api/usuarios	Crear usuario.
Usuarios	PUT	http://localhost:3000/api/usuarios/:id	Actualizar usuario.
Usuarios	DELETE	http://localhost:3000/api/usuarios/:id	Eliminar lógicamente usuario.
Salas	GET	http://localhost:3000/api/salas	Listar salas.
Salas	POST	http://localhost:3000/api/salas	Crear sala.
Salas	PUT	http://localhost:3000/api/salas/:id	Actualizar sala.
Salas	DELETE	http://localhost:3000/api/salas/:id	Eliminar lógicamente sala.
Reservas	GET	http://localhost:3000/api/reservas	Listar reservas.
Reservas	POST	http://localhost:3000/api/reservas	Crear reserva.
Reservas	PATCH	http://localhost:3000/api/reservas/:id/estado	Cambiar estado de una reserva.
Reservas	GET	http://localhost:3000/api/reservas/disponibilidad	Consultar disponibilidad de una sala.

10. COMANDOS UTILIZADOS DURANTE EL DESARROLLO

Durante el desarrollo del frontend se utilizaron comandos en terminal para crear el proyecto, instalar dependencias y levantar el servidor local.

Acción	Comando utilizado	Descripción
Ir a la carpeta del usuario	cd C:\Users\rosita	Permite ubicarse en la carpeta donde se creó el proyecto frontend.
Crear proyecto React con Vite	npm.cmd create vite@latest reserva_salas2_front -- --template react	Crea el proyecto React usando Vite.
Entrar al proyecto frontend	cd C:\Users\rosita\reserva_salas2_front	Permite ingresar a la carpeta del frontend.
Instalar dependencias base	npm.cmd install	Instala las dependencias iniciales del proyecto.
Instalar Bootstrap y React Router	npm.cmd install bootstrap react-router-dom	Agrega Bootstrap para diseño responsivo y React Router DOM para navegación.
Levantar frontend	npm.cmd run dev	Ejecuta el frontend en entorno local.
Crear carpetas internas	New-Item -ItemType Directory -Force src\components, src\pages, src\services	Crea las carpetas principales del frontend.
Entrar al backend	cd C:\Users\rosita\reservas_salas2_mvc	Permite ingresar a la carpeta del backend.
Levantar backend	npm.cmd run dev	Ejecuta el backend en http://localhost:3000.

11. PUERTOS UTILIZADOS

Proyecto	Puerto	URL
Frontend React	5173	http://localhost:5173
Backend Node.js / Express	3000	http://localhost:3000

Para probar correctamente el sistema, ambos servidores deben estar levantados al mismo tiempo:

Terminal	Proyecto	Comando
Terminal 1	Frontend	npm.cmd run dev
Terminal 2	Backend	npm.cmd run dev

12. ERRORES Y DIFICULTADES ENCONTRADAS DURANTE EL DESARROLLO

Durante el desarrollo se presentaron algunos errores propios del proceso de integración entre frontend y backend. Estos errores fueron revisados y corregidos durante las pruebas.

Error o situación	Causa identificada	Solución aplicada
PowerShell bloqueó el comando npm	Windows bloqueó la ejecución del script npm.ps1.	Se utilizó npm.cmd en lugar de npm.
Pantalla blanca en React	Faltaba o estaba mal escrito export default App.	Se corrigió el cierre de App.jsx usando export default App.
Error Apps is not defined	Se escribió Apps en vez de App.	Se corrigió el nombre correcto del componente.
Error por archivo mal nombrado	Un archivo quedó como Reservas_jsx en vez de Reservas.jsx.	Se renombró correctamente el archivo.
Redirección al login	El token JWT había expirado o no estaba disponible.	Se volvió a iniciar sesión y se validó el token en localStorage.
Ruta directa del backend mostraba "Token no proporcionado"	Se intentó acceder a rutas protegidas desde el navegador sin token.	Se realizó la prueba desde el frontend, enviando el token en el encabezado Authorization.
Usuario eliminado seguía apareciendo	El backend realiza eliminación lógica.	Se comprobó que el usuario cambia a estado inactivo.
Sala eliminada seguía apareciendo	El backend realiza eliminación lógica.	Se comprobó que la sala cambia a estado no_disponible.
Reservas quedaban en estado pendiente	El backend crea reservas inicialmente como pendientes.	Se agregó cambio de estado mediante botones para confirmar, finalizar o cancelar.
Advertencia de Chrome sobre contraseña	Chrome detectó admin123 como contraseña común.	Se identificó que era una alerta del navegador y no un error del sistema.
Problemas visuales al actualizar datos	Algunas acciones requerían recargar o actualizar datos desde el backend.	Se ajustaron las funciones para volver a consultar los registros después de crear, editar o eliminar.

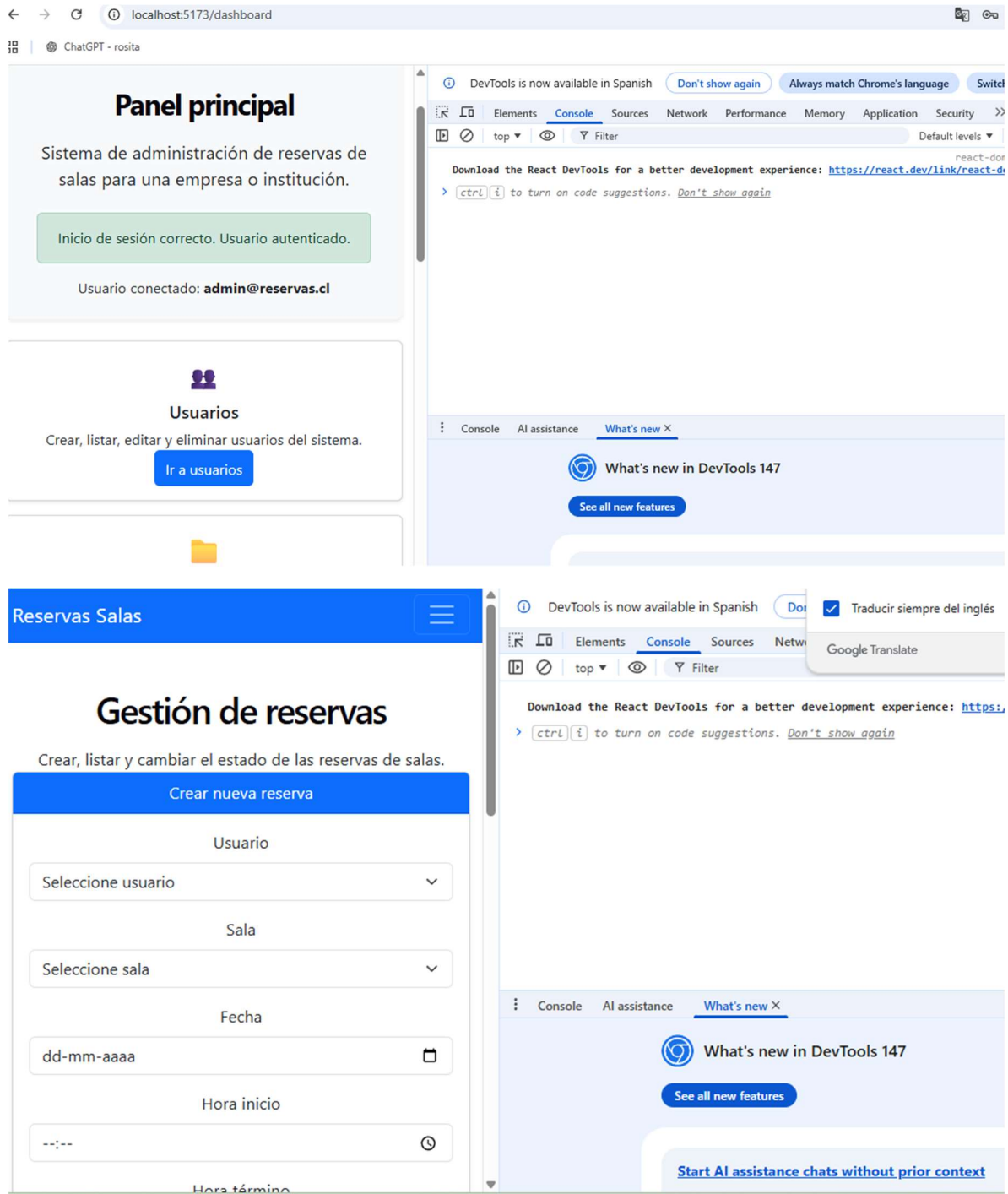
13. REVISION DISEÑO RESPONSIVO

Durante el desarrollo del frontend se revisó el comportamiento responsivo de la aplicación utilizando las herramientas de desarrollo del navegador Google Chrome.

Para esta revisión se activó la vista de dispositivos móviles mediante DevTools, verificando que las páginas principales del sistema se adaptaran correctamente a distintos tamaños de pantalla.

Las vistas revisadas fueron:

Vista revisada	Resultado observado
Login	La pantalla de inicio de sesión se adapta correctamente, manteniendo el logo, texto institucional y formulario de acceso ordenados.
Dashboard	Las tarjetas del panel principal se reorganizan verticalmente, permitiendo una navegación clara desde dispositivos móviles.
Reservas	El formulario de creación de reservas se adapta a una sola columna, manteniendo visibles los campos de usuario, sala, fecha, hora y observación.
Navbar	El menú principal se transforma en botón hamburguesa, permitiendo acceder a las rutas del sistema en pantallas pequeñas.



Esta revisión permitió comprobar que el frontend mantiene una estructura visual ordenada tanto en escritorio como en dispositivos móviles, cumpliendo con el requerimiento de una interfaz responsiva.